Python Notes: Weight Conversion Exercise 🏋️

Convert Kilograms ↔ Pounds Using `if-elif-else`

Video: Python weight conversion exercise 🏋️
Video URL: https://www.youtube.com/watch?v=80kjCBRjkmU
Level: Beginner
Prerequisites: Variables, `input()`, type casting, arithmetic operators, `if-elif-else`, and f-strings.

I. Introduction

A **weight converter** is a Python program that converts a person's weight from one unit of measurement into another.

In this project, the user enters their weight and selects the unit in which it is currently measured. The program then converts the value and displays the result in the other unit.

The two supported units are:

- **Kilograms (`kg`)** — commonly used for measuring body mass.
- **Pounds (`lbs`)** — another unit used for measuring body weight.

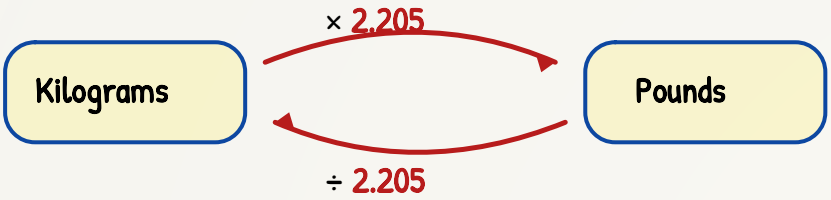
Example

If a person weighs 180 pounds, the program converts that value into approximately 81.6 kilograms.

If a person weighs 81 kilograms, the program converts it into approximately 178.6 pounds.

2. Understanding the Conversion Formulas

Before writing the program, understand the mathematical formulas.



2.1 Kilograms to Pounds

To convert kilograms into pounds, multiply the weight in kilograms by approximately `2.205`.

Formula:

$$\text{Pounds} \approx \text{Kilograms} \times 2.205$$

Example: Convert 81 kilograms into pounds.

$$81 \times 2.205 = 178.605$$

Rounded to one decimal place:

$$178.6 \text{ lbs}$$

Python expression:

```
pounds = kilograms * 2.205
```

Example: Convert 180 pounds into kilograms.

$$180 \div 2.205 \approx 81.633$$

Rounded to one decimal place:

81.6 kg

Python expression:

```
kilograms = pounds / 2.205
```

Quick reference table

Conversion	Formula	Example
Kilograms → Pounds	`weight * 2.205`	81 kg ≈ 178.6 lbs
Pounds → Kilograms	`weight / 2.205`	180 lbs ≈ 81.6 kg

MOST IMPORTANT

The video uses `2.205` as an approximation. The more precise relationship is `1 kg = 2.2046226218... lbs`. Using `2.205` is sufficient for this beginner exercise, but small rounding differences may occur.

3. Step 1: Accept the User's Weight

First, create a variable named `weight` and ask the user to enter their weight.

```
weight = input("Enter your weight: ")
```

Correct approach

```
weight = float(input("Enter your weight: "))
```

Explanation

- `input()` accepts the user's entry.
- `float()` converts a valid numeric string into a floating-point number.
- The result is stored in the `weight` variable.

Example:

```
weight = float("180")

print(weight)
print(type(weight))
```

Output:

```
180.0
<class 'float'>
```



MOST IMPORTANT

Use `float()` because weight can contain decimal values, such as `81.5` kilograms.

5. Step 3: Use an `if-elif-else` Statement

The program needs to make a decision based on the selected unit.

Recall the purpose of each conditional keyword:

Keyword	Purpose
`if`	Checks the first condition.
`elif`	Checks another condition if the preceding condition is false.
`else`	Handles cases where none of the preceding conditions is true.

Our conversion logic will be:

1. If the user enters `K`, convert kilograms to pounds.
2. If the user enters `L`, convert pounds to kilograms.
3. Otherwise, display an invalid-unit message.

Basic structure

```
if unit == "K":
    # Convert kilograms to pounds
elif unit == "L":
    # Convert pounds to kilograms
else:
    # Display an invalid-unit message
```



MOST IMPORTANT

Use `==` to compare values. A single `=` is used for assignment.

For example, if `weight` is `81.0`:

```
weight = 81.0
weight = weight * 2.205

print(weight)
```

Output:

```
178.605
```

The variable `weight` is reassigned to contain the converted value.

To make the code shorter, we can use the multiplication assignment operator `*=`:

```
weight *= 2.205
```

Both versions perform the same operation here.

7. Step 5: Convert Pounds to Kilograms

When the user enters `L`, the program should divide the current weight by `2.205`.

```
elif unit == "L":
    weight = weight / 2.205
```

For example:

```
weight = 180.0
weight = weight / 2.205

print(weight)
```

8. Step 6: Store the New Unit

After converting the weight, the program should indicate the unit of the result.

For the output to be clear:

- If the original unit is kilograms, the new unit is pounds (`lbs`).
- If the original unit is pounds, the new unit is kilograms (`kgs`).

We can store the new unit in a separate variable called `new_unit` .

```
if unit == "K":
    weight *= 2.205
    new_unit = "lbs"

elif unit == "L":
    weight /= 2.205
    new_unit = "kgs"
```

Why use a separate variable?

Without the `new_unit` variable, the program would need additional conditions when printing the result.

By storing the output unit alongside the converted weight, we can display the result using one f-string.

Example:

```
print(f"Your converted weight is: {weight} {new_unit}")
```

This is easier to maintain and read.

Example:

```
weight = 81.632653

print(round(weight, 1))
```

Output:

```
81.6
```

Another example:

```
weight = 178.605

print(round(weight, 1))
```

Output:

```
178.6
```

We can round the variable directly:

```
weight = round(weight, 1)
```

MOST IMPORTANT

`round(weight, 1)` rounds the numeric value to one decimal place. It does not guarantee that a trailing zero will be displayed. If you need exactly one decimal place in the output, use an f-string format such as `{weight:.1f}`.

Example output:

```
Is this weight in kilograms or pounds? (K/L): pizza
pizza is not a valid unit.
```

However, we also need to ensure that the program does not continue displaying a conversion result after an invalid entry.

To solve this, place the result-printing statement inside the valid branches, or use a Boolean variable to indicate whether a conversion succeeded.

The simpler option for this exercise is to print the result inside each valid branch.

II. Complete Beginner Program

Here is the complete version following the video's approach, with improved handling of invalid units.

Sample execution 3: Decimal input

```
Enter your weight: 65.5
Is this weight in kilograms or pounds? (K/L): K
Your weight is 144.4 lbs.
```

Sample execution 4: Invalid unit

```
Enter your weight: 180
Is this weight in kilograms or pounds? (K/L): pizza
pizza is not a valid unit.
```

Notice that the program does not print a converted weight when the unit is invalid.

12. An Improved Version: Accept Lowercase and Uppercase Input

The original video requires capital letters `K` and `L`. We can improve the program by accepting either uppercase or lowercase letters.

The `upper()` string method converts a string to uppercase.

```
unit = input("Enter the unit (K/L): ").upper()
```

Now the following inputs are all accepted as expected:

- `K`
- `k`
- `L`
- `l`

This version introduces two useful ideas:

- `.strip()` removes extra whitespace around the input.
- `.upper()` makes the unit selection case-insensitive.

FREQUENTLY ASKED

Note: This program assumes the weight entered is a valid numeric value. Entering text such as `hello` for the weight would raise a `ValueError`. You can add `try-except` handling in a more advanced version.

13. Common Mistakes to Avoid

Mistake 1: Forgetting type casting

Incorrect:

```
weight = input("Enter your weight: ")
weight = weight * 2.205
```

The input is a string, so it cannot be used directly in this numeric calculation.

Correct:

```
weight = float(input("Enter your weight: "))
```

Mistake 2: Reversing the formulas

Incorrect for kilograms to pounds:

```
weight = weight / 2.205
```

Mistake 4: Forgetting indentation

Incorrect:

```
if unit == "K":
weight *= 2.205
```

Correct:

```
if unit == "K":
    weight *= 2.205
```

Mistake 5: Printing the result even after invalid input

If the user enters an invalid unit, do not display a converted result. Print the result only after a successful conversion.

Mistake 6: Displaying the wrong unit

If you convert kilograms to pounds, the output must be labelled `lbs`. If you convert pounds to kilograms, label the output `kgs` or `kg`.

The numeric value alone is not enough; the unit must also be correct.

---

14. Practice Questions

Try answering these questions before checking the answer key.

Question 4

```
unit = "X"

if unit == "K":
    print("Kilograms")
elif unit == "L":
    print("Pounds")
else:
    print("Invalid unit.")
```

Question 5

What is the difference between these two expressions?

```
weight = float(input("Enter weight: "))
```

```
weight = input("Enter weight: ")
```

Answer key

Question	Answer
1	176.4
2	100.0
3	Convert to pounds.
4	Invalid unit.
5	The first stores a floating-point number; the second stores a string.

Challenge 2: Round to two decimal places

Change the output so that it displays two decimal places.

Hint:

```
print(f"Your converted weight is {weight:.2f} {new_unit}.")
```

Challenge 3: Validate the numerical input

Prevent the program from crashing when someone enters text instead of a number.

Hint: Use `try-except` with `float()`.

Challenge 4: Add a repeat option

Ask the user whether they want to convert another weight. If they answer yes, repeat the program using a loop.

This will introduce the `while` loop in a practical context.

---

16. Last-Minute Revision Sheet

Important formulas

Conversion	Formula
Kilograms to pounds	`weight * 2.205`
Pounds to kilograms	`weight / 2.205`

Concept	Purpose
`.upper()`	Converts a string to uppercase.
`.strip()`	Removes surrounding whitespace.
F-string	Displays the converted value and its unit.

Five key points to remember

1. Use `float(input(...))` to accept decimal weight values.
2. Multiply by `2.205` to convert kilograms to pounds.
3. Divide by `2.205` to convert pounds to kilograms.
4. Use `if-elif-else` to choose the correct formula and handle invalid units.
5. Display the correct output unit and round the value to an appropriate number of decimal places.

Final takeaway

This weight converter is a practical beginner project that combines user input, type casting, arithmetic operations, conditional statements, rounding, and formatted output.

Once you understand how the two conversion formulas work and why each conditional branch is necessary, try writing the program from memory. Then extend it to support lowercase inputs, error handling, and repeated conversions to strengthen your Python programming skills.